Architectural Blueprint and Research Plan: Generating Clinical Trials, CRFs, and CQL via Natural Language and HL7 FHIR R4

The intersection of artificial intelligence and clinical informatics presents an unprecedented opportunity to automate the translation of unstructured clinical knowledge into computable artifacts. Historically, the design of clinical trials, the authoring of Case Report Forms (CRFs), and the codification of clinical decision support logic have required immense manual effort, often leading to delays in research execution and data interoperability challenges. The Health Level Seven (HL7) Fast Healthcare Interoperability Resources (FHIR) Release 4 (R4) standard provides a robust, graph-oriented data model capable of representing the entire clinical research lifecycle[1]. However, the syntactic complexity of FHIR R4 and the intricacies of Clinical Quality Language (CQL) create a steep learning curve for clinical researchers. The conceptualization of an application capable of generating clinical trials, CRFs, and CQL directly from natural language requires a profound understanding of multiple FHIR R4 domains, the Structured Data Capture (SDC) implementation guide, and the integration of Large Language Models (LLMs) via the Model Context Protocol (MCP)[3]. This comprehensive analysis deconstructs the FHIR R4 data model specific to research and clinical reasoning, establishing a highly detailed structural framework. Furthermore, it outlines an exhaustive research plan for developing an LLM-driven architecture that bridges the semantic gap between natural language protocols and executable FHIR resources.

Domain 1: Modeling Clinical Trials and Cohorts in FHIR R4

The foundation of any clinical research application within the FHIR ecosystem relies on the robust representation of the study protocol, the participating entities, and the inclusion or exclusion criteria. In FHIR R4, the research domain is driven primarily by the ResearchStudy and ResearchSubject resources, augmented by the Group resource for eligibility definition[5]. Generating these interconnected resources from a natural language text requires an artificial intelligence system to map unstructured narratives into highly constrained, terminology-bound JSON structures.

The ResearchStudy Resource and Protocol Representation

The ResearchStudy resource serves as the canonical backbone for study protocol definition. It represents a scientific investigation designed to generate health-related knowledge, encompassing both interventional clinical trials and observational, retrospective, or

prospective studies[5]. An application synthesizing a trial from natural language must accurately map unstructured text into the structured elements of the ResearchStudy resource. A natural language prompt such as "A Phase III, double-blind study of drug X in adults with Type 2 Diabetes" contains multiple distinct entities that must be systematically extracted and appropriately categorized within the FHIR schema.

The critical attributes of the ResearchStudy resource include the status, phase, and condition elements. The status element captures the recruitment and operational state of the trial, utilizing values such as active, recruiting, suspended, or completed[5]. The phase element, captured via the ResearchStudyPhase terminology binding, dictates the maturity of the clinical investigation, such as Phase II or Phase III trials[5]. The condition element defines the disease or problem being studied and is typically bound to standardized ontologies such as SNOMED CT or ICD-10[9]. An LLM must be engineered to extract these entities from natural language and populate the corresponding fields accurately, ensuring that the generated data adheres strictly to the defined value sets.

Furthermore, the protocol element provides a critical linkage within the FHIR ecosystem by referencing a PlanDefinition resource, which dictates the step-by-step execution of the study[6]. This highlights the graph-oriented nature of FHIR; a single resource rarely exists in isolation. To ensure interoperability with external registries, the generative application must also produce ResearchStudy resources that align with established global mappings, such as the ClinicalTrials.gov and BRIDG 5.1 data models[8].

FHIR R4 ResearchStudy Element	**Data Type**	**ClinicalTrials.gov / Protocol Mapping**	**Natural Language Extraction Target**
identifier	Identifier	NCT Number / Protocol ID	"Protocol number: EXO-2024-001"
title	string	Official Study Title	"A randomized trial of novel interventions..."
status	code	Overall Recruitment Status	"Currently enrolling participants"
phase	CodeableConcept	Study Phase (e.g., Phase 2)	"Phase IIa investigation"
condition	CodeableConcept	Conditions / Diagnoses	"Patients diagnosed with acute asthma"

enrollment	Reference(Group)	Eligibility Criteria / Cohort	"Adults over 18 with no prior cardiac history"

In addition to the core fields, the ResearchStudy resource supports extensive customization through extensions. For instance, the SiteRecruitment extension is utilized when studies need to find sites according to specific criteria, such as requiring a freezer capable of very low temperatures or specific imaging equipment[9]. The application must possess the intelligence to recognize when a natural language prompt implies the need for such extensions and inject them seamlessly into the generated JSON output.

Step-by-Step Guide: Creating a ResearchStudy from Scratch

To programmatically generate a ResearchStudy from scratch, the following step-by-step process and strict data types must be applied[5]:

1. **Instantiate the Resource**: Define the root JSON object with "resourceType": "ResearchStudy".
2. **Assign Identifiers & Status**: Provide a business identifier (Data Type: Identifier) to track the study across systems. A status is mandatory (Data Type: code, constrained to values like draft, active, completed, or withdrawn)[5].
3. **Define Title and Phase**: Set the human-readable title (Data Type: string) and the study phase (Data Type: CodeableConcept, bound to the ResearchStudyPhase value set, such as "Phase II" or "Phase III")[5].
4. **Specify Conditions**: Define the target diseases or problems using the condition element (Data Type: CodeableConcept, preferably leveraging terminologies like SNOMED CT or ICD-10)[5].
5. **Establish the Cohort (Group)**: Create an independent Group resource to capture the actual inclusion and exclusion criteria. Link it back to the study using the enrollment element (Data Type: Reference(Group))[5].
6. **Link the Protocol**: Associate the actionable execution workflow by using the protocol element to reference a pre-defined plan (Data Type: Reference(PlanDefinition))[5].

Cohort Definition and Computable Eligibility Criteria

The transition from a natural language eligibility criterion to a computable FHIR representation represents one of the most complex tasks for an LLM agent. In FHIR R4, the criteria for recruitment are predominantly managed through the Group resource, which defines a collection of entities that share specific characteristics[8]. The application must parse complex inclusion and exclusion criteria, such as "Patients must have an HbA1c greater than 7.0% and no prior history of myocardial infarction," and construct a Group resource containing a precise set of characteristic definitions. Eligibility criteria linked by logical operations like "OR" can be represented by utilizing multiple FHIR Group resources to define alternative enrollment

pathways[12].
For more advanced computable criteria, the Group resource operates in tandem with the EvidenceVariable resource, which specifies exactly what data elements are of interest for the cohort definition[8]. When eligibility rules require complex temporal or logical relationships—such as identifying patients who have experienced three specific adverse events within a six-month rolling window—the architecture mandates the use of Clinical Quality Language (CQL). This logic is wrapped in a Library resource, which is subsequently referenced by the cohort definition[13]. The generative application must therefore be capable of determining the optimal technical pathway: deciding whether a natural language criterion can be satisfied by a simple Group.characteristic (such as gender or an age range) or if it requires the synthesis of a discrete CQL script to execute a temporal query against a patient's longitudinal record.

Participant Tracking and Anonymization via ResearchSubject

Once the trial is defined and the cohort is established, the ResearchSubject resource connects an actual Patient to the ResearchStudy[8]. This resource tracks the progression of the individual through the trial using the status element, capturing states such as candidate, eligible, active, or withdrawn[8]. The resource also utilizes the progress element to delineate specific milestones, recording the start and end dates for states like screening or on-study periods[8].
Crucially, for clinical research applications prioritizing patient privacy, the ResearchSubject record contains an internal pointer to the Patient resource, allowing the application to maintain anonymized trial data while preserving the ability to securely query the underlying clinical record[8]. The Patient resource itself may contain identifiable information, but the ResearchSubject acts as a proxy, identified by an arbitrary study-specific identifier. The generative application must understand this separation of concerns, ensuring that when instructed to construct participant tracking mechanisms, it correctly instantiates ResearchSubject records that reference, rather than duplicate, patient demographic data. Furthermore, the application must manage consent by linking the ResearchSubject to a Consent resource, detailing the specific LOINC codes that govern the participant's agreement to be involved in the research[8].

Domain 2: Formulating Case Report Forms (CRFs) with Structured Data Capture

Clinical trials rely heavily on Case Report Forms (CRFs) to capture participant data precisely, consistently, and securely. In the FHIR R4 standard, these forms are represented using the Questionnaire resource, while the structured data collected from participants or clinicians is stored in the corresponding QuestionnaireResponse resource[15]. To autonomously generate these forms from natural language instructions, the application must leverage the advanced capabilities defined in the Structured Data Capture (SDC) Implementation Guide, which significantly expands the utility of the base FHIR specification[17].

Questionnaire Architecture and Item Hierarchy

A Questionnaire resource is a recursive structure composed primarily of item elements, which can be categorized as groups, displays, or specific questions[16]. An LLM analyzing a natural language input—such as "Create a clinical intake form to collect patient demographics, including a required field for date of birth and an optional field for smoking status"—must generate a deeply nested JSON structure. This structure must assign appropriate item.type values, such as date, choice, string, or boolean, based on the semantic intent of the requested question[20].

The recursive nature of the item element allows for complex logical organization, grouping related questions into coherent sections[16]. For example, a "Vital Signs" group might contain nested items for systolic blood pressure, diastolic blood pressure, and heart rate. The generative model must recognize these implicit groupings in natural language and reflect them in the hierarchical tree of the JSON output.

Step-by-Step Guide: Creating a Questionnaire (CRF) from Scratch

Generating a complete Questionnaire (CRF) involves constructing a recursive hierarchical structure using specific data types[16].

1. **Initialize the Resource**: Define the root object with "resourceType": "Questionnaire".
2. **Set Metadata**: Assign a canonical url (Data Type: uri), a status (Data Type: code, e.g., draft or active), and a title (Data Type: string) to identify the form context.
3. **Construct the Item Hierarchy**: The core of the form is the item array (Data Type: BackboneElement). Every nested question or grouping must exist within this array. Each item MUST have a unique linkId (Data Type: string) and a type (Data Type: code)[16].
4. **Assign Data Types for Questions**: The item.type dictates the expected answer format[19]. Essential types include:
    - *Simple inputs:* string, text (for multi-line text), boolean, integer, decimal[19].
    - *Time-based:* date, dateTime, time[19].
    - *Selection/Structure:* choice, open-choice (requires nested answerOption or answerValueSet elements), quantity (requires UCUM-coded units), reference (points to another FHIR resource), and attachment[19].
    - *Organization:* group (used to nest child items) and display (read-only instructional text)[19].
5. **Apply SDC Extensions (Optional)**: Inject extensions for advanced form behavior, such as item.enableWhen for conditional logic, or definitionExtract to map answers directly to other FHIR resources during extraction[22].

Leveraging SDC Extensions for Form Behavior and Rendering

The Structured Data Capture (SDC) Implementation Guide enhances the base Questionnaire resource by introducing specific extensions for advanced rendering, form behavior, and complex logical calculations[18]. For example, complex conditional logic, such as "Only ask for the

number of cigarettes consumed per day if the patient identifies as a current smoker," is implemented using the enableWhen element natively available in FHIR[23]. However, if the requisite logic exceeds the capabilities of simple equality checks provided by enableWhen, the application must generate FHIRPath or CQL expressions and embed them using SDC behavioral extensions to calculate dynamic form states[23].

To achieve semantic interoperability across distinct healthcare systems, the generated Questionnaire must bind its questions and permissible answer options to standard clinical terminologies. The LLM must be explicitly instructed to map natural language concepts to Logical Observation Identifiers Names and Codes (LOINC) or Systematized Nomenclature of Medicine (SNOMED CT) codes. This is accomplished using the item.code element for the question itself and the answerValueSet or answerOption elements for the predefined responses[2]. Ensuring that a generated form uses standard codes rather than arbitrary strings is what allows downstream analytical systems to compare trial data reliably.

Advanced Data Extraction Mechanisms

The ultimate clinical and analytical value of a FHIR-native CRF lies in its ability to seamlessly transform unstructured or semi-structured questionnaire responses into standard clinical resources, such as an Observation, Condition, or MedicationStatement[15]. This capability prevents trial data from becoming siloed indefinitely within QuestionnaireResponse resources, allowing it to integrate directly into the longitudinal patient record. The generative application must produce Questionnaire resources that include precise extraction metadata. The SDC guide specifies multiple methodologies for this extraction process, primarily Observation-based extraction, Definition-based extraction, and Template-based extraction[22].

Extraction Strategy	Implementation Mechanism	Optimal Clinical Use Case	Generative LLM Requirements
Observation-based	Relies on item.code (e.g., LOINC) and the observationExtract extension. Automatically generates Observation resources.	Simple clinical measurements (e.g., vital signs, lab results, single-value survey responses).[27]	The LLM must accurately map questions to specific LOINC codes and inject the observationExtract extension at the item level.
Definition-based	Uses the definitionExtract extension to map specific item	Complex resource generation and attribute updating (e.g., modifying	The LLM must possess deep knowledge of all FHIR resource

	answers to discrete target resource paths (e.g., Patient.birthDate).	Patient demographics, creating a ServiceRequest).[22]	schemas to generate accurate FHIRPath pointers for mapping.
Template-based	Utilizes a skeleton target resource embedded within the Questionnaire, substituting predefined variables from the response.	Highly complex, multi-resource extraction requiring transactional bundles and explicit relational mapping.[29]	The LLM must generate precise structural templates and appropriately use the extractAllocateId extension to maintain referential integrity across the bundle.

The generative application must process the user's natural language intent to determine the optimal extraction strategy for any given form. For instance, a prompt requesting an "Adverse Event reporting form" mandates that the LLM utilize definition-based or template-based extraction to map the clinician's responses accurately into a FHIR AdverseEvent resource. By embedding this mapping data directly into the Questionnaire, the system ensures that the QuestionnaireResponse/$extract operation will function flawlessly on the target FHIR server, automatically generating a transactional bundle of relevant resources[17].

When executing definition-based extraction, the generative model must carefully define the extraction context using a canonical URL representing the base FHIR type or a specific profile, ensuring that elements like Patient.name.given are perfectly aligned with the schema[22]. The complexity of ensuring that these paths align perfectly with the target resource structures represents a significant challenge that the LLM must be trained to navigate seamlessly.

Domain 3: Executable Clinical Logic and Decision Support

Beyond static data capture, modern clinical trials and sophisticated clinical decision support (CDS) systems require executable logic to automate workflows, trigger alerts, calculate clinical quality measures, and determine patient eligibility dynamically. The FHIR Clinical Reasoning module provides the foundational framework for this execution, utilizing the Library, PlanDefinition, and ActivityDefinition resources in close conjunction with Clinical Quality Language (CQL)[31].

Clinical Quality Language (CQL) and the Library Resource

Clinical Quality Language (CQL) is an HL7 standard that provides a high-level, human-readable

yet entirely computable domain-specific language for expressing clinical knowledge[29]. Generating CQL directly from natural language is a highly complex undertaking because it requires the LLM to possess a dual understanding: it must comprehend the clinical intent of the user and master the underlying hierarchical FHIR data model. For example, translating the natural language phrase "Identify patients with an active diagnosis of diabetes who have not had an HbA1c test in the past year" into CQL requires the LLM to define a precise patient context, retrieve Condition resources matching a specific diabetes value set, retrieve Observation resources for HbA1c tests constrained by a specific temporal interval, and perform a logical evaluation to ensure the absence of the latter[30].

Once the CQL is successfully generated by the language model, it cannot be executed directly in its raw text format by most FHIR evaluation engines. It must be translated into a machine-readable canonical representation known as the Expression Logical Model (ELM)[35]. The application's architecture must incorporate a discrete backend compilation step—utilizing robust tools like the Java-based CQL-to-ELM translator—to convert the LLM-generated CQL into an ELM XML or JSON format[33].

This compiled logic is subsequently encapsulated within a FHIR Library resource. The Library resource functions as a distribution container for knowledge artifacts. The raw CQL string is stored as a base64-encoded string under the text/cql content type, while the compiled ELM is stored simultaneously as application/elm+xml or application/elm+json[37]. This dual-storage mechanism ensures that the clinical logic remains perfectly human-readable for clinical review and auditing purposes, while remaining instantly executable by a CQL engine, such as the cql-execution library running on a Node.js server or the fqm-execution library[36]. When a FHIR server evaluates this library, the engine decodes the base64 payload, processes the ELM, and evaluates it against a bundle of patient data retrieved from the server[37].

Step-by-Step Guide: Creating CQL and Executable Logic from Scratch

Writing and packaging Clinical Quality Language (CQL) from scratch requires strict adherence to syntax, terminology bindings, and FHIR type mappings[41].

1. **Declare Library and Data Model**: Begin the CQL text file by declaring the library name/version and the target data model[41]. Example: library MyTrialLogic version '1.0.0' followed by using FHIR version '4.0.1'[36].
2. **Include Helpers**: Include FHIRHelpers to facilitate automatic type conversions between FHIR primitives and CQL primitives[34]. Example: include FHIRHelpers version '4.0.1' called FHIRHelpers
3. **Define Terminology**: Explicitly define the code systems and value sets required for your logic[41]. Example: codesystem "SNOMEDCT": 'http://snomed.info/sct'[34].
4. **Set Context**: Define the execution context to evaluate the logic against individual patient records. Example: context Patient[36].
5. **Map Data Types**: Ensure your logic aligns CQL system types with FHIR data types[42]. For instance:
    - System.Boolean maps to FHIR.boolean

    - System.String maps to FHIR.string
    - System.DateTime maps to FHIR.dateTime
    - Interval<System.DateTime> maps to FHIR.Period[42].
6. **Write Evaluation Statements**: Create define statements to query resources based on the defined terminology.
   Example: define "Has Diabetes": exists([Condition: "DiabetesCodes"]).
7. **Compile to ELM**: Raw CQL cannot be executed directly by FHIR engines. Use a CQL-to-ELM translator to compile the human-readable CQL text into the machine-readable Expression Logical Model (ELM) JSON format[34].
8. **Package into a FHIR Library Resource**: Create a FHIR Library resource to distribute the logic[39]. Set "type": {"coding": [{"code": "logic-library"}]}. In the content array (Data Type: Attachment), embed the logic by providing base64-encoded strings for both the raw CQL (setting "contentType": "text/cql") and the compiled ELM (setting "contentType": "application/elm+json")[37].

Orchestrating Workflows with PlanDefinition and ActivityDefinition

While the Library resource contains the raw mathematical and logical expressions, the PlanDefinition resource orchestrates exactly how, when, and to whom that logic is applied. A PlanDefinition represents a pre-defined group of actions to be taken under particular circumstances, acting as the connective tissue for clinical protocols, order sets, and automated workflows[43].
An application tasked with generating clinical trials must construct a PlanDefinition to represent the study's Schedule of Activities. The resource utilizes three primary components to achieve this orchestration:

1. **TriggerDefinition:** Defines the specific event that initiates the evaluation of the clinical logic. This can be a scheduled time, a continuous data change monitor, or a named event, such as a CDS Hooks order-sign or order-select event[37].
2. **Condition:** Specifies whether the subsequent action should actually take place. The condition element references a specific boolean expression located within the linked CQL Library[37].
3. **Action:** Details the specific activity to be performed if the condition evaluates to true. The action frequently references an ActivityDefinition resource—which might define an instruction to order a specific lab test or administer a medication—or it may trigger a Questionnaire to be rendered in the provider's user interface[37].

By autonomously generating a tightly coupled network consisting of Library, PlanDefinition, and ActivityDefinition resources, the LLM transforms a static text protocol into an active, event-driven monitoring agent. When applied to a specific patient context via the $apply operation, the PlanDefinition evaluates its logic and generates a CarePlan or a RequestGroup containing the specific, actionable directives for that individual patient[43].

Domain 4: Large Language Model Integration and

Natural Language Translation

The core value proposition of the proposed application is its profound ability to interpret unstructured natural language and accurately emit highly structured, schema-compliant FHIR R4 resources and CQL scripts. Standard, unmodified LLMs face significant limitations in this specific domain. Their reliance on static, potentially outdated training data often results in severe hallucinations, persistent syntactic errors, and a failure to comply with the strict cardinality and structural rules mandated by the FHIR schema[3]. Furthermore, navigating the graph-based structure of FHIR, where resources are deeply interconnected by references rather than flat tables, challenges the traditional reasoning capabilities of language models[47].
To overcome these inherent limitations, the application must employ an advanced architectural paradigm involving Retrieval-Augmented Generation (RAG), the Model Context Protocol (MCP), and highly iterative validation pipelines.

Retrieval-Augmented Generation and the Model Context Protocol (MCP)

To ensure that the LLM generates accurate and interoperable FHIR resources, its reasoning must be strictly grounded in the official HL7 FHIR specifications, published Implementation Guides (IGs), and standardized global terminologies. A sophisticated Retrieval-Augmented Generation (RAG) architecture allows the system to retrieve exact schema definitions and terminology value sets (such as SNOMED CT, LOINC, and RxNorm) at runtime, injecting this context directly into the LLM's prompt window before generation occurs[7].
The dynamic integration between the LLM and the target FHIR server is facilitated by the Model Context Protocol (MCP). The MCP acts as a secure, declarative, and standardized bridge, allowing the LLM agent to dynamically query the FHIR server for patient data, retrieve existing resource profiles, and interact with external terminology servers[4]. By providing the LLM with read-only access to the schema constraints via MCP prior to resource generation, the system drastically reduces the likelihood of producing invalid resource structures[11]. Furthermore, when generating executable logic, the MCP enables the LLM to verify that the specific data elements it intends to reference in its CQL queries actually exist within the target server's specific data model, preventing the generation of logic that fails upon execution[4].

Iterative Syntactic and Semantic Validation Pipelines

The autonomous generation of complex FHIR resources, particularly those involving deeply nested hierarchies like the Questionnaire or heavily constrained workflow resources like the PlanDefinition, requires a multi-stage generation and validation loop. Empirical studies analyzing the generation of FHIR from natural language indicate that zero-shot prompting yields significantly lower syntactic validity compared to one-shot or few-shot prompting methodologies[50].
The application's architecture must implement a dedicated "Validator Agent" to oversee the generation process. Once the primary LLM generates a preliminary JSON structure, the

Validator Agent submits this output to a standard, deterministic FHIR validation engine. This could be the widely used HAPI FHIR Validator or the FHIR $validate-code operation linked to a dedicated terminology server like Ontoserver[50].
If the validation engine returns a 400 Bad Request or an OperationOutcome detailing specific schema violations—such as a missing mandatory field, an invalid terminology binding, or a reference inconsistency—the Validator Agent parses the technical error message, constructs a corrective natural language prompt, and instructs the primary LLM to regenerate the resource[50]. This iterative, "LLM-in-the-loop" validation continues autonomously until the output achieves absolute syntactic and semantic compliance with the FHIR R4 standard.

Intermediate Representation Strategies

Because generating perfectly formatted, deeply nested JSON strings is inherently difficult for LLMs, the application architecture will heavily utilize intermediate representation formats. Instead of prompting the LLM to generate raw FHIR JSON directly from natural language, the system will prompt the LLM to generate a simplified YAML representation or leverage an intermediate Domain Specific Language (DSL) such as FHIR Shorthand (FSH). The backend application will then use deterministic, rule-based compilers (such as SUSHI for FSH) to transform the intermediate representation into the final, compliant FHIR JSON artifacts. This hybrid approach intelligently leverages the LLM's strength in natural language understanding and semantic mapping while relying on deterministic software to handle the strict syntactical formatting of the final JSON schema.

Comprehensive Research and Development Plan

To realize this advanced application, a highly structured, multi-phase research and development plan is required. The plan emphasizes a methodical progression from basic ontology mapping and infrastructure configuration to the deployment of autonomous, MCP-enabled LLM agents capable of executing end-to-end clinical workflow generation.

Phase 1: Knowledge Graph and Profiling Infrastructure Configuration

The primary objective of the initial phase is to establish the absolute ground truth for the LLM. The generative application cannot construct compliant resources without a pristine, instantly accessible repository of FHIR profiles and terminologies.

1. **Ingestion of FHIR R4 Specifications:** Deploy a high-performance vector database to comprehensively index the core HL7 FHIR R4 documentation, the Structured Data Capture (SDC) Implementation Guide, and the Clinical Reasoning Module documentation[18]. This creates the foundational knowledge base for the RAG pipeline.
2. **Terminology Server Integration:** Integrate a syndicated terminology server, such as Ontoserver, to provide rapid runtime validation of SNOMED CT, LOINC, and RxNorm codes via the standard $validate-code operation[2].
3. **Profile Definition and Mapping:** Establish the base validation templates for the ResearchStudy, ResearchSubject, and Group resources. Ensure these profiles demonstrate strict alignment with global data standards like ClinicalTrials.gov and BRIDG

5.1 mappings to guarantee downstream interoperability[16].

Phase 2: LLM Agent Framework and Prompt Pipeline Design

Phase 2 focuses entirely on constructing the generative core of the application, utilizing the Model Context Protocol (MCP) and sophisticated multi-agent orchestration.

1. **RAG Pipeline Development:** Engineer the retrieval pipeline to extract highly relevant schema snippets based on the user's natural language input. If the user requests a CRF, the pipeline must autonomously retrieve the Questionnaire schema and the relevant SDC behavioral extensions to inject into the LLM's context window[4].
2. **Prompt Engineering and Few-Shot Optimization:** Develop exhaustive libraries of few-shot examples for each supported resource type. For example, provide the LLM with curated, validated examples of natural language eligibility criteria mapped flawlessly to Group and EvidenceVariable resources[6].
3. **Iterative Validation Loop Integration:** Build the Validator Agent architecture. Implement an automated feedback loop where generated JSON is tested continuously against the HAPI FHIR Validator, with technical errors autonomously translated into corrective prompts and fed back to the LLM[4].

Phase 3: Form Builder and Data Extraction Engine

This phase addresses the translation of natural language into dynamic, data-capturing CRFs and ensuring the resultant data is thoroughly interoperable.

1. **Questionnaire Generation Capability:** Train the language model to map conversational requests into distinct Questionnaire items, accurately assigning data types and utilizing the enableWhen element to implement complex skip logic[16].
2. **Extraction Mapping Engine:** Develop the logic required for the LLM to append definition-based and observation-based extraction extensions, such as sdc-questionnaire-definitionExtract and observationExtract, to the generated forms[29].
3. **Simulation of $extract Operation:** Create a testing sandbox to simulate the generation of a QuestionnaireResponse by a fictitious user, and subsequently simulate the execution of the $extract operation. Verify that the target Observation or Condition resources are correctly instantiated in the test FHIR server[16].

Phase 4: CQL Compilation and Workflow Automation

The most technically demanding phase involves generating executable logic and orchestrating it via the Clinical Reasoning module.

1. **CQL Generation via Natural Language:** Fine-tune the LLM to translate complex clinical conditions and temporal logic into valid CQL syntax. Implement intermediate validation using a discrete CQL syntax parser to prevent the generation of hallucinated functions or invalid data types[33].
2. **Automated ELM Compilation Pipeline:** Integrate the Java-based CQL-to-ELM translator directly into the backend architecture. Create an automated pipeline that takes the LLM-generated CQL, compiles it to ELM JSON, and embeds both payloads into a

base64-encoded Library resource[56].

3. **PlanDefinition Orchestration:** Develop the capability for the LLM to generate overarching PlanDefinition resources that correctly reference the compiled Library in their condition elements, and logically link to appropriate ActivityDefinition resources to trigger the desired workflows[51].

Development Phase	Key Architectural Deliverables	Success Validation Metrics
Phase 1: Infrastructure	Terminology Server, Vector DB, Core FHIR Profiles.	100% successful $lookup and $validate-code responses during unit testing.
Phase 2: LLM Framework	RAG Pipeline, MCP Integration, Validator Agent.	>95% syntactic validity on first-pass JSON generation[59].
Phase 3: SDC / CRFs	Questionnaire generation, enableWhen logic, Extraction Extensions.	Zero data loss or referential errors during the simulated $extract operation into standard clinical resources.
Phase 4: Executable Logic	CQL generation, CQL-to-ELM translation, PlanDefinition orchestration.	Successful execution of the generated Library against a test Patient bundle using a standard execution engine like cql-execution.
Phase 5: Real-World Test	End-to-end platform user interface and API deployment.	Successful, autonomous translation of an entire unstructured clinical trial protocol into a deployable FHIR package.

Phase 5: Validation, Security, and Real-world Testing

The final phase ensures the comprehensive system is robust, highly secure, and ready for deployment in a live clinical environment.

1. **End-to-End Workflow Testing:** Process existing, publicly available clinical trial protocols through the application. Audit the generated ResearchStudy, Questionnaire, Library, and

PlanDefinition resources for absolute semantic alignment with the original text[2].

2. **Security and Authorization Integration:** Implement SMART on FHIR authorization protocols and strict OAuth 2.0 scopes to ensure that the LLM agent only accesses and generates resources within permitted, patient-approved boundaries[61].
3. **Human-in-the-Loop Review Interfaces:** Design a sophisticated user interface that presents the generated FHIR artifacts—rendered as human-readable forms or intuitive workflow diagrams—to the clinical researcher for final approval and electronic signature before any data is committed to the production FHIR server[4].

Conclusion

The realization of an application capable of generating clinical trials, Case Report Forms, and computable clinical logic directly from natural language represents a monumental paradigm shift in clinical informatics. By deeply integrating Large Language Models with the strict structural boundaries of the HL7 FHIR R4 standard, the proposed architecture effectively mitigates the traditional bottlenecks associated with manual protocol codification. The precise synthesis of the ResearchStudy resource for protocol definition, the SDC-enhanced Questionnaire for data capture, and the PlanDefinition backed by CQL for executable logic creates a closed-loop system where unstructured clinical intent becomes a dynamic, interoperable, and computable reality. Executing the comprehensive multi-phase research plan outlined above will yield a highly sophisticated system that not only accelerates the setup of clinical trials but ensures the resulting data is natively standardized, highly structured, and immediately actionable within the global healthcare ecosystem.

Bibliografia

1. hl7.fhir.r4.examples | ResearchStudy - SIMPLIFIER.NET, https://simplifier.net/packages/hl7.fhir.r4.examples/4.0.1/files/95246
2. A standards-based approach to digital health research: implementing the people heart study, https://pmc.ncbi.nlm.nih.gov/articles/PMC12646378/
3. FHIR-RAG-MEDS: Integrating HL7 FHIR with Retrieval-Aug- mented Large Language Models for Enhanced Medical Deci - arXiv, https://arxiv.org/pdf/2509.07706
4. Enhancing Clinical Decision Support and EHR Insights through LLMs and the Model Context Protocol: An Open-Source MCP-FHIR Framework - arXiv, https://arxiv.org/html/2506.13800v1
5. ResearchStudy - FHIR v6.0.0-ballot4, https://build.fhir.org/researchstudy.html
6. FHIR Resources for Clinical Research, https://www.devdays.com/wp-content/uploads/2021/12/DD21US_20210607_Hugh_Glover_FHIR_Resources_for_Clinical_Research.pdf
7. ResearchStudy - FHIR v6.0.0-ballot4, https://build.fhir.org/researchstudy-definitions.html
8. Accessing ClinicalTrials.gov Data in the About HL7® FHIR® Format, https://clinicaltrials.gov/data-api/fhir

9. Research Study - NCPI FHIR Implementation Guide, https://nih-ncpi.github.io/ncpi-fhir-ig/research_study.html
10. FHIR R4 Resource Questionnaire - FHIRTools.net, https://fhirtools.net/fhir_r4_resource_questionnaire.html
11. ResearchStudy - FHIR v6.0.0-ballot4, https://build.fhir.org/researchstudy-mappings.html
12. Implementation of a Clinical Trial Recruitment Support System Based on Fast Healthcare Interoperability Resources (FHIR) in a Cardiology Department - ResearchGate, https://www.researchgate.net/publication/360864070_Implementation_of_a_Clinical_Trial_Recruitment_Support_System_Based_on_Fast_Healthcare_Interoperability_Resources_FHIR_in_a_Cardiology_Department
13. (PDF) Fast Healthcare Interoperability Resources (FHIR) for Interoperability in Health Research: A Systematic Review (Preprint) - ResearchGate, https://www.researchgate.net/publication/360708025_Fast_Healthcare_Interoperability_Resources_FHIR_for_Interoperability_in_Health_Research_A_Systematic_Review_Preprint
14. Home - Common CQL Artifacts for FHIR (US-Based) v2.0.0-cibuild, https://build.fhir.org/ig/HL7/us-cql-ig/
15. QuestionnaireResponse - FHIR v6.0.0-ballot4, https://build.fhir.org/questionnaireresponse.html
16. FHIR Questionnaire Resource: Structure, QuestionnaireResponse & How It Works - Medblocks Blog, https://medblocks.com/blog/fhir-questionnaire-essentials-how-to-get-started
17. Questionnaire response extract to resource(s) - JSON Representation - FHIR specification, https://build.fhir.org/ig/HL7/sdc/en/OperationDefinition-QuestionnaireResponse-extract.json.html
18. SDC Base Questionnaire - Structured Data Capture v4.0.0 - FHIR specification, https://build.fhir.org/ig/HL7/sdc/en/StructureDefinition-sdc-questionnaire-definitions.html
19. FHIR Questionnaire Item Types: All Question & Answer Types Explained with Examples, https://medblocks.com/blog/a-beginners-guide-to-fhir-questionnaire-question-types
20. PlanDefinition - FHIR v4.0.1, http://fhir.outburn.co.il/R4-spec/plandefinition-profiles.html
21. PlanDefinition - FHIR v6.0.0-ballot4, https://build.fhir.org/plandefinition-definitions.html
22. Understanding SDC Definition-Based Resource Extraction in FHIR, https://fhirblog.com/2026/02/26/definition-based-resource-extraction/
23. Release Notes | Aidbox Docs - Health Samurai, https://www.health-samurai.io/docs/aidbox/overview/release-notes
24. Advanced Behavior Questionnaire - Structured Data Capture v4.0.0-ci-build, https://build.fhir.org/ig/HL7/sdc/en/StructureDefinition-sdc-questionnaire-behave

-definitions.html
25. Advanced Rendering Questionnaire - Structured Data Capture v4.0.0 - FHIR specification, https://build.fhir.org/ig/HL7/sdc/en/StructureDefinition-sdc-questionnaire-render-definitions.html
26. Building Healthcare Forms in Flutter: Meet FHIR Renderer Questionnaire - Medium, https://medium.com/@cristian.apr99/building-healthcare-forms-in-flutter-meet-fhir-renderer-questionnaire-96cf0e5c29e3
27. Form Data Extraction - Structured Data Capture v4.0.0-ci-build - FHIR specification, https://build.fhir.org/ig/HL7/sdc/en/extraction.html
28. How to extract data from forms | Formbox Docs - Health Samurai, https://www.health-samurai.io/docs/formbox/aidbox-ui-builder-alpha/form-creation/how-to-guides/how-to-extract-data-from-forms
29. CQL Specification - Clinical Quality Language Specification v1.5.3 - HL7.org, https://cql.hl7.org/
30. Understanding Clinical Quality Language (CQL) | by Ryan Lindbeck - Medium, https://rycharlind.medium.com/understanding-clinical-quality-language-cql-620783f71a7e
31. Clinicalreasoning-module - FHIR v6.0.0-ballot4, https://build.fhir.org/clinicalreasoning-module.html
32. FHIR Measures for Implementers - HL7 FHIR DevDays, https://www.devdays.com/wp-content/uploads/2024/07/6.12.24-Rene-Spronk-FHIR-Measures-for-Implementers.pdf
33. GitHub - cqframework/clinical_quality_language: Clinical Quality Language (CQL) is an HL7 specification for the expression of clinical knowledge that can be used within both the Clinical Decision Support (CDS) and Clinical Quality Measurement (CQM) domains. This repository contains complementary tooling in support of that specification., https://github.com/cqframework/clinical_quality_language
34. V. LLM-in-the-Loop CQL execution with unstructured data and FHIR terminology support, https://nuchange.ca/2025/06/v-llm-in-the-loop-cql-execution-with-unstructured-data-and-fhir-terminology-support.html
35. Clinical Quality Language and CQL Engines: The Basics - NCQA, https://www.ncqa.org/resources/clinical-quality-language-and-cql-engines-the-basics/
36. Decision Support Logic - WHO SMART Trust v1.1.4, https://smart.who.int/trust/1.1.4/decision_support.html
37. Knowledge Representation for PDDI CDS - Potential Drug-Drug Interaction (PDDI) Clinical Decision Support v1.0.0 - FHIR specification, https://build.fhir.org/ig/HL7/PDDI-CDS/knowledge-artifacts.html
38. QMs - Quality Measure Implementation Guide v5.0.0 - FHIR specification, https://build.fhir.org/ig/HL7/cqf-measures/measure-conformance.html
39. HL7.FHIR.UV.CMI\Conformance - FHIR v4.0.1 - FHIR specification, https://build.fhir.org/ig/HL7/Content-Management-Infrastructure-IG/artifact-conf

ormance.html
40. projecttacoma/fqm-execution: fqm-execution is a library that allows users to calculate FHIR-based electronic Clinical Quality Measures (eCQMs) and retrieve the results in a variety of formats · GitHub, https://github.com/projecttacoma/fqm-execution
41. Decision Support Logic - WHO SMART Trust v1.2.0, https://smart.who.int/trust/1.2.0/decision_support.html
42. HL7.FHIR.UV.CMI\Using CQL - FHIR v4.0.1 - FHIR specification, https://build.fhir.org/ig/HL7/Content-Management-Infrastructure-IG/using-cql.html
43. PlanDefinition - FHIR v6.0.0-ballot4, https://build.fhir.org/plandefinition.html
44. Care Planning and Management using FHIR patient care and protocol resources - FHIR DevDays, https://www.devdays.com/wp-content/uploads/2021/12/Rene-Spronk-Care-Planning-and-Management-_-DevDays-2019-Amsterdam-1.pdf
45. Plandefinition-operation-apply - FHIR v4.0.1, http://fhir.outburn.co.il/R4-spec/plandefinition-operation-apply.html
46. question answering on patient medical records with private fine-tuned llms - arXiv, https://arxiv.org/pdf/2501.13687
47. [2509.19319] FHIR-AgentBench: Benchmarking LLM Agents for Realistic Interoperable EHR Question Answering - arXiv, https://arxiv.org/abs/2509.19319
48. FHIR-AgentBench: Benchmarking LLM Agents for Realistic Interoperable EHR Question Answering - arXiv, https://arxiv.org/html/2509.19319v1
49. Health Ecosystem WorkFlow Prototyping (FHIR®©) HTHeco © Beta 1.8, https://fireitup.azurewebsites.net/
50. Assessing the Potential of an LLM-Powered System for Enhancing FHIR Resource Validation - PubMed, https://pubmed.ncbi.nlm.nih.gov/40380578/
51. A Validation Tool (VaPCE) for Postcoordinated SNOMED CT Expressions: Development and Usability Study - JMIR Medical Informatics, https://medinform.jmir.org/2025/1/e67984
52. Home - CH ORF (R4) v3.0.2 - fhir.ch, https://fhir.ch/ig/ch-orf/index.html
53. Datatypes - FHIR v5.0.0 - HL7 FHIR Specification, https://fhir.hl7.org/fhir/datatypes.html
54. Questionnaire - HIEBus™ FHIR Interface, https://www.fhir.docs.careevolution.com/interface/r4/resources/questionnaire.html
55. CQL - Clinical Quality Language - Connect | eCQI Resource Center, https://ecqi.healthit.gov/cql/connect
56. BECarePlan - JSON Representation - HL7 Belgium Patient Care v1.1.0 - Ehealth, https://www.ehealth.fgov.be/standards/fhir/patient-care/StructureDefinition-be-care-plan.profile.json.html
57. Enhancing Dementia Risk Mitigation: Standardized Digital Monitoring in Clinical Trials and Platform Design for At-Risk Elderly Individuals - FH Joanneum, https://epub.fh-joanneum.at/obvfhjoa/download/pdf/10108285
58. PSS Provider System Capability Statement - JSON Representation,

https://www.ehealth.fgov.be/standards/fhir/pss/CapabilityStatement-PSSProviderCapabilityStatement.json.html
59. ehealth-library - TTL Representation - eHealth Infrastructure v3.5.1, https://ehealth.sundhed.dk/fhir/3.5.1/StructureDefinition-ehealth-library.profile.ttl.html
60. FHIR as a Unifying Format for Genomic Research Data Tracking, Aggregation, and Integration - bioRxiv, https://www.biorxiv.org/content/10.64898/2025.12.22.695544v1.full.pdf
61. FHIR R4 supported resource types for HealthLake - AWS Documentation, https://docs.aws.amazon.com/healthlake/latest/devguide/reference-fhir-resource-types.html
62. FHIR $questionnaire-package operation for HealthLake - AWS Documentation, https://docs.aws.amazon.com/healthlake/latest/devguide/reference-fhir-operations-questionnaire-package.html